

2. CONFIGURARAREA ȘI UTILIZAREA MODULELOR DE TIP TIMER

1. Scopul lucrării

Lucrarea își propune prezentarea circuitelor de tip timer prezente în microcontrolerul de tip ARM Cortex M0+ MKL46Z256VLL4.

Se detaliază programarea și utilizarea circuitului PIT (periodic interrupt timer), precum și folosirea modului de generare a semnalelor PWM.

2. Breviar teoretic

2.1.Circuite timer din MKL46Z256VLL4

Microprocesorul conține patru circuite de tip timer, prezentate în următorul tabel:

Module	Descriere
Modulul Timer/PWM (TPM)	• Modul de ceas selectabil TPM • Prescalare divizat prin 1, 2, 4, 8, 16, 32, 64, sau 128 • Numărător pe 16 biți sau modul de numărător cu numărare în sus sau în jos • Șase canale configurabile pentru captura de intrare, compararea de la ieșire, sau alinierea marginii • Modul PWM • Suportă generarea unei întreruperi și / sau a unei cereri DMA, pe un canal • Suportă generarea unei întreruperi și / sau a unei cereri DMA când se termină numărarea
Timere de întreruperi periodice (PIT)	• Timer de întrerupere cu un scop general, • Contoare de întrerupere pentru activarea conversiei ADC • Rezoluție numărătorului de 32 biți • Cronometrat de magistrala frecvenței de ceas
Timer de mica putere (LPTMR)	• Contor de timp de 16 biți sau contor de impulsuri cu comparare • Sursă de ceas configurabilă pentru filtru prescaler / glitch • Sursă de intrare configurabilă pentru contor de impulsuri

Numărător în timp real (RTC)	• Numărare pe 16 biți în sus • Modul pe 16 biți la limita potrivită • Software-ul de întrerupere periodic controlabil la potrivire • Software-ul sursei de ceas selectabilă de intrare a prescaler-ului cu prescaler programabil pe 16 biți • XOSC 32.678 kHz frecvență nominală
------------------------------	--

2.2.Modulul Timer/PWM (TPM)

Această secțiune prezintă modul în care modulul a fost configurat în chip.

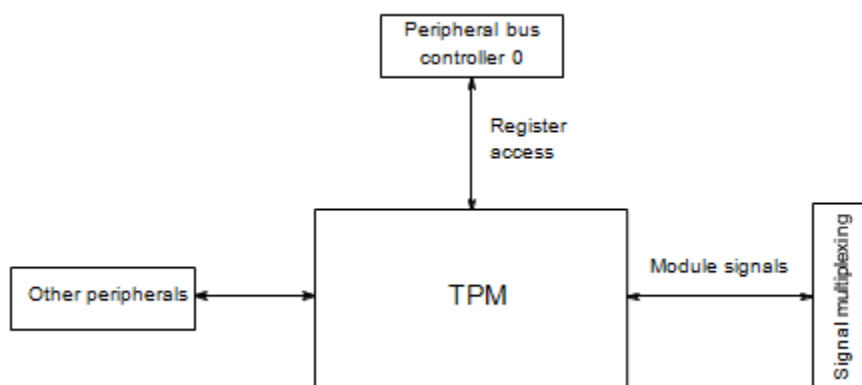


Figura 1. Configurarea TPM

2.2.1. Informații privind instantierea TPM

Acest dispozitiv conține trei module TPM de putere scăzută (TPM). Toate modulele TPM de pe dispozitiv sunt configurate doar ca funcție TPM de bază, nu acceptă funcția de decodor în cuadratură, și toate pot fi functionale în modul Stop / VLPS. Sursa ceasului este fie externa fie internă în modul Stop / VLPS.

Tabelul de mai jos arată modul în care sunt configurate aceste module.

Tabelul 2. Configuratia modulului TPM

TPM	Număr de canale	Caracteristici / Utilizare
TPM0	6	TPM de bază, funcțional în modul Stop / VLPS
TPM1	2	TPM de bază, funcțional în modul Stop / VLPS
TPM2	2	TPM de bază, funcțional în modul Stop / VLPS

Există mai multe conexiuni spre și către TPM, având scopul de a ajuta cumparatorii.

2.2.2. Opțiunile ceasului

Blocurile TPM sunt cronometrate de un singur ceas TPM, care poate fi ales dintre OSCERCLK, MCGIRCLK, MCGPLLCLK / 2, sau MCGFLLCLK. Sursa selectată este controlată de SIM_SOPT2 [TPMSRC] și SIM_SOPT2 [PLLFLLSEL].

De asemenea fiecare TPM sprijină un modul de ceas extern (TPM_SC [CMOD] = 1x), în care contorul sporește, după o marja de creștere sincronizată (cu sursa de ceas TPM aleasă) detectată de o intrare de ceas externă. Ceasul extern disponibil (TPM_CLKIN0 sau TPM_CLKIN1) este selectat de registrul de control SIM_SOPT4 [TPMxCLKSEL]. Pentru a garanta funcționarea valabilă, ceasului extern selectat trebuie să fie mai mic decât jumătate din frecvența sursei ceasului TPM selectat.

2.2.3. Opțiuni de declanșare

Fiecare TPM are o sursă de intrare, selecție și declanșare controlată de domeniul [TRGSEL] TPMx_CONF pentru a utiliza, pentru început, contorul și / sau reîncărcarea acestuia. Opțiunile disponibile sunt prezentate în tabelul următor.

Tabelul 3. Opțiuni de declanșare TPM

TPMx_CONF[TRGSEL]	Sursa selectată
0000	External trigger pin input (EXTRG_IN)
0001	CMP0 output
0010	Reserved
0011	Reserved
0100	PIT trigger 0
0101	PIT trigger 1
0110	Reserved
0111	Reserved
1000	TPM0 overflow
1001	TPM1 overflow
1010	TPM2 overflow
1011	Reserved
1100	RTC alarm
1101	RTC seconds
1110	LPTMR trigger
1111	Reserved

2.2.4. Baza de timp globală

Fiecare TPM are o funcție de bază de timp globală controlată de bitul TPM_x_CONF [GTBEEN]. TPM1 este configurat ca timp global, atunci când această opțiune este activată.

2.2.5. Întreruperi TPM

TPM are mai multe surse/cauze de întrerupere. Cu toate acestea, aceste surse sunt împreună OR'd pentru a genera o singură cerere de întrerupere către controlerul de întrerupere. Atunci când are loc o întrerupere TPM, se citesc registrele de stare TPM pentru a determina sursa exactă de întrerupere.

2.3. Timere de întreruperi periodice (PIT)

Această secțiune prezintă modul în care modulul a fost configurat în chip.

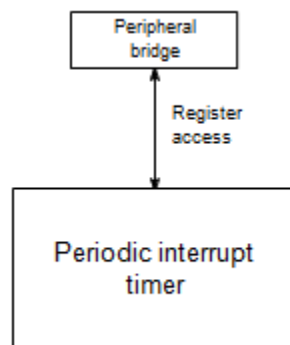


Figura 2. Configurarea PIT

2.3.1. Sarcini periodice de declansare PIT / DMA

PIT generează evenimente de declanșare periodice la canalul mux DMA așa cum este prezentat în tabelul de mai jos.

Tabelul 4. Sarcinile canalului PIT pentru declansarea periodica a DMA

Canalul PIT	Numărul canalului DMA
PIT Canalul 0	DMA Canalul 0
PIT Canalul 1	DMA Canalul 1

2.3.2. Declanșările PIT / ADC

Declanșările PIT sunt selectate ca surse de declanșare ADC_x folosind biții SOPT7[ADC_xTRGSEL] în modulul SIM.

2.3.3. Declanșările PIT / TPM

Declanșările PIT sunt selectate ca surse de declanșare TPMx folosind biții TPMx_CONF[TRGSEL] în modulul TPM.

2.3.4. Declanșările PIT / CAD

Canalul 0 PIT este configurat ca sursă de declanșare hardware DAC

2.4.Timer de mică putere (LPTMR)

Această secțiune prezintă modul în care modulul a fost configurat în chip.

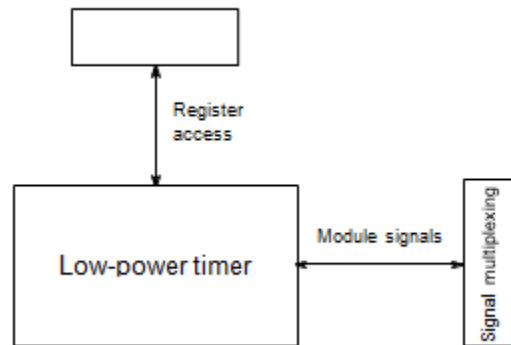


Figura 3. Configurarea LPTMR

2.4.1. Informații de instantiere LPTMR

Timer-ul de mică putere (LPTMR) permite operarea în timpul tuturor modurilor de putere.LPTMR, poate funcționa ca o întrerupere în timp real sau acumulator de puls. Acesta include un prescaler 2N (modul de întrerupere în timp real) sau filtru glitch (modul de acumulator de puls).

LPTMR poate fi cronometrat de la ceasul intern de referință, intern 1 kHz LPO, OSCERCLK, sau un cristal extern de 32.768 kHz. În modul VLLS0, opțiunea de cronometrare este limitată la un pin extern cu un OSC configurat pentru operațiunea bypass (ceas extern).

O întrerupere este generată (și contorul se poate reseta), în cazul în care contorul egalează valoarea de 16 biți din registrul de comparare.

2.4.2. Opțiuni de introducere a contorului de puls LPTMR

LPTMR_CSR [TPS] configurează sursa de intrare utilizată în modul contorului de puls. Tabelul de mai jos prezintă sarcinile specifice de intrare a cipului pentru acest domeniu.

LPTMR_CSR[TPS]	Numărul de intrare al contorului de puls	Intrarea în cip
00	0	CMP0 output
01	1	LPTMR_ALT1 pin

10	2	LPTMR_ALT2 pin
11	3	LPTMR_ALT3 pin

2.4.3. Opțiuni de cronometraj ale PTMR Prescaler / filtru glitch

Prescalerul și filtrul glitch al modulului LPTMR poate fi cronometrat de la unul din cele patru surse determinate de LPTMR0_PSR .

NOTĂ!

Ceasul ales trebuie să rămână activat în cazul în care LPTMR trebuie să continue să funcționeze în toate modurile de mică putere necesare.

2.5. Numărător în timp real (RTC)

Această secțiune prezintă modul în care modulul a fost configurat în chip.

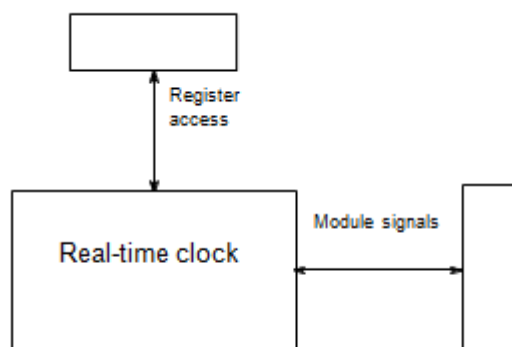


Figura 4. Configurarea RTC

2.5.1. Informații de instanțiere RTC

Prescalerul RTC este cronometrat de ERCLK32K. RTC este resetat numai pe POR. RTC_CR [OSCE] poate suprascrive configurația sistemului OSC , configurand OSC pentru funcționarea la 32 kHz frecvența cristalului în toate modurile de alimentare, cu excepția VLLS0, precum și prin orice resetare a sistemului. Când OSCE este activat, RTC suprascrive configurațiile de condensatoare.

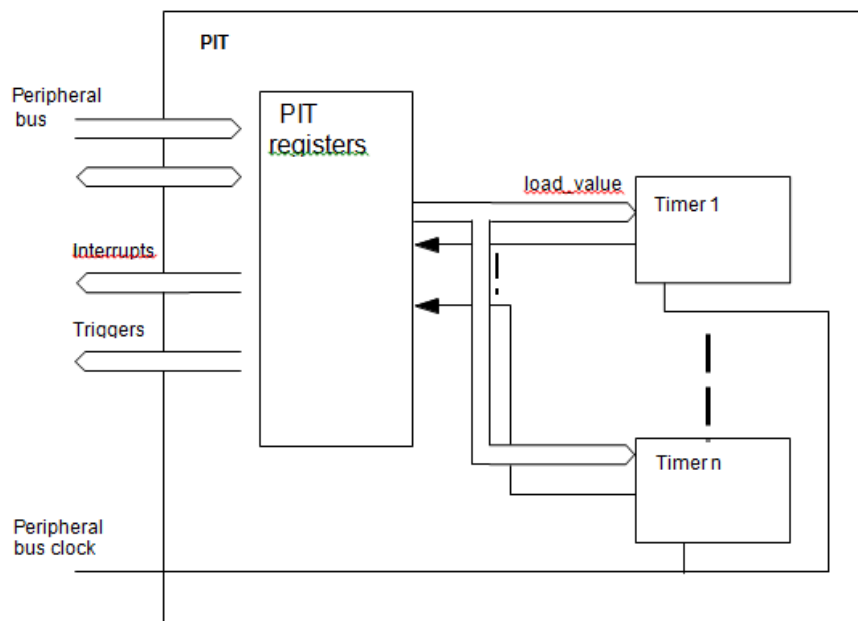
2.5.2. Opțiunile RTC_CLKOUT

Pinul RTC_CLKOUT poate fi acționat fie cu iesirea RTC de 1 Hz fie cu sursa de ceas OSCERCLK de pe cip. Controlul acestei opțiuni se face prin bitul SIM_SOPT2 [RTCCLKOUTSEL] .

Când RTCCLKOUTSEL = 0, ceasul de iesire RTC 1 Hz este selectat pe pinul RTC_CLKOUT. Când RTCCLKOUTSEL = 1, ceasul de iesire OSCERCLK este selectat pe pinul RTC_CLKOUT.

2.6. Programarea modului PIT

2.6.1. Diagrama bloc

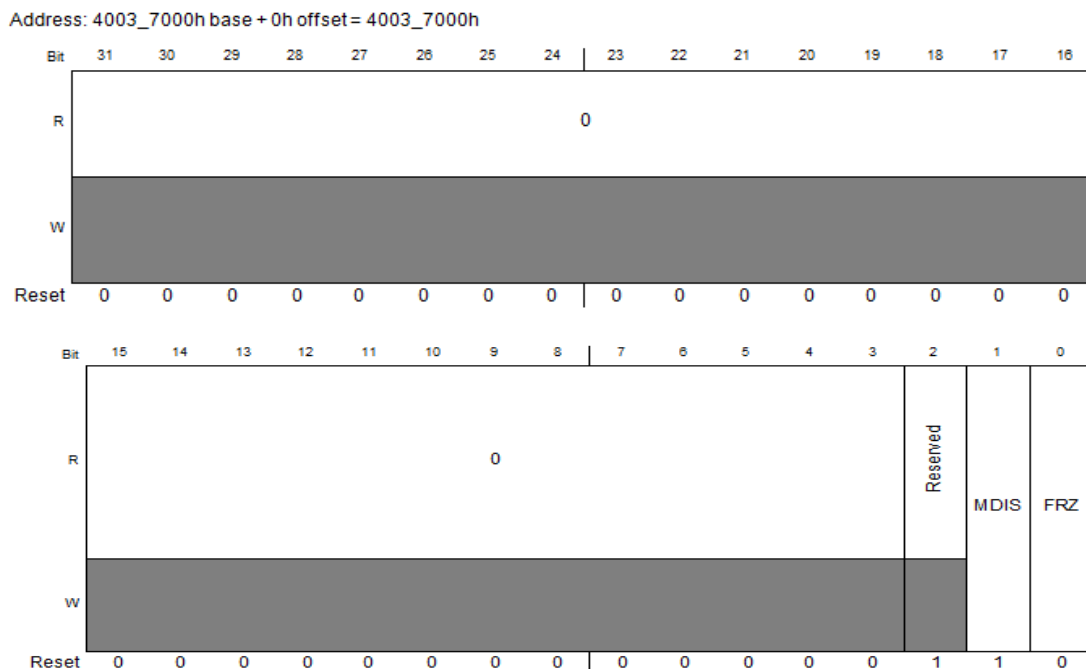


2.6.2. Descrierea regiștrilor

Adresa in hex	Numele registrului	Lățimea (în biți)	Acces	Valoarea de reset
4003_7000	PIT Module Control Register (PIT_MCR)	32	R/W	0000_0006h
4003_70E0	PIT Upper Lifetime Timer Register (PIT_LTMR64H)	32	R	0000_0000h
4003_70E4	PIT Lower Lifetime Timer Register (PIT_LTMR64L)	32	R	0000_0000h
4003_7100	Timer Load Value Register (PIT_LDVAL0)	32	R/W	0000_0000h
4003_7104	Current Timer Value Register (PIT_CVAL0)	32	R	0000_0000h
4003_7108	Timer Control Register (PIT_TCTRL0)	32	R/W	0000_0000h
4003_710C	Timer Flag Register (PIT_TFLG0)	32	R/W	0000_0000h
4003_7110	Timer Load Value Register (PIT_LDVAL1)	32	R/W	0000_0000h
4003_7114	Current Timer Value Register (PIT_CVAL1)	32	R	0000_0000h
4003_7118	Timer Control Register (PIT_TCTRL1)	32	R/W	0000_0000h
4003_711C	Timer Flag Register (PIT_TFLG1)	32	R/W	0000_0000h

2.6.3. Registrul modulul PIT de control (PIT_MCR)

Acest registru activează sau dezactivează ceasurile timerului PIT și controlează timerele atunci când PIT intră în modul Debug.

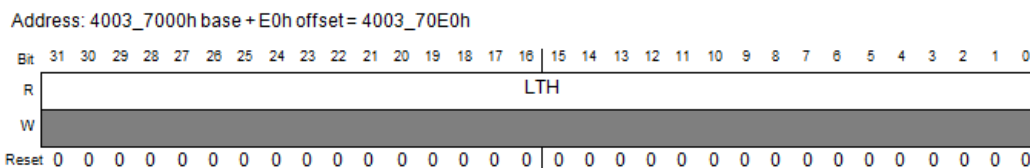


2.6.4. Decrierea domeniului PIT_MCR

Domeniul	Decriere
31–3 Reserved	Acest domeniu este rezervat Acest domeniu doar de citire este rezervat si are întotdeauna valoarea 0
2 Reserved	Acest domeniu este rezervat
1 MDIS	Modul de dezactivare - (secțiunea PIT) Dezactivează timeri standard. Acest domeniu trebuie activat inaintea oricărei alte setări. 0 Clock pentru timeri standard PIT este activat. 1 Clock pentru timeri PIT este dezactivat.
0 FRZ	Freeze Permite timerelor sa fie oprite când dispozitivul este in modul Debug. 0 Timerele continua să ruleze in modul Debug. 1 Timerele sunt oprite in modul Debug.

2.6.5. Registrul timer-ului PIT cu durata de viata maximă (PIT_LTMR64H)

Acest registru este destinat pentru aplicații care leagă timer 0 si timer 1 pentru a construi un lifetimer de 64 biti.



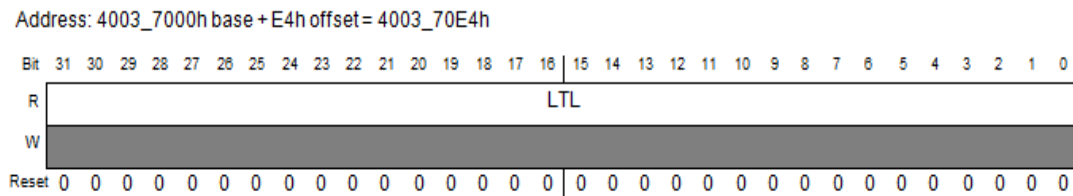
2.6.6. Descrierea PIT_LTMR64H

Domeniul	Descrierea
31–0 LTH	Valoarea de durată de viață a timer-ului Arată valoarea timer a timerului 1. Dacă acest registru este citit la un moment t1, LTMR64L prezinta valoarea timer 0 în momentul t1.

2.6.7. Registrul timer-ului PIT cu durata de viata minimă(PIT_LTMR64L)

Acest registru este destinat aplicațiilor care leagă timer 0 și timer 1 pentru a construi un timer pe 64 biți

Pentru a utiliza LTMR64H și LTMR64L, timer-ul 0 și timer-ul 1 trebuie să fie legate. Pentru a obține valoarea corectă, mai întâi se citește LTMR64H și apoi LTMR64L. LTMR64H va avea valoarea de CVAL1 la data de primul acces, LTMR64L va avea valoarea de CVAL0 la primul acces, prin urmare, aplicatia nu are nevoie să își facă să își facă probleme în legătură cu efectele transferului la rularea numărătorului.

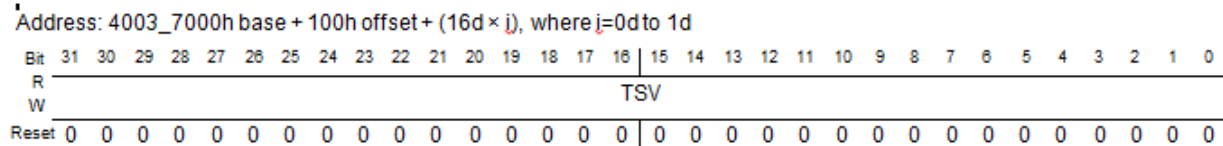


2.6.8. Descrierea PIT_LTMR64L

Domeniul	Descrierea
31–0 LTL	Valoarea de durată de viață a timer-ului Arată valoarea timer a timerului 1. Dacă acest registru este citit la un moment t1, LTMR64H prezinta valoarea timer 0 în momentul t1.

2.6.9. Registrul timer-ului de încărcare a valorii (PIT_LDVALn)

Acești regiștri selectează perioada de timeout pentru timer-ul de întreruperi.



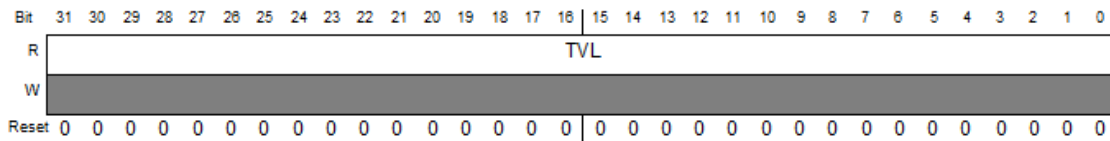
2.6.10. Decrierea PIT_LDVALn

Domeniul	Decriere
31–0 TSV	Valoarea de start a timer-ului Setează valoarea de start a timer-ului. Timer-ul va număra inapoi până ajunge la 0, apoi va genera o întrerupere și va încărca valoarea în registru din nou. Scriind o nouă valoare în acest registru nu va resteta timer-ul.ș în schimb valoarea va fi încărcată după ce timer-ul expiră. Pentru a renunța la ciclul curent și pentru a incepe o perioadă a timer-ului cu o nouă valoare, timer-ul trebuie sa fie dezactivat și activat din nou.

2.6.11. Valoarea curentă a timer-ului registrul (PIT_CVALn)

Acești regiștri indică poziția curentă a timer-ului.

Address: 4003_7000h base + 104h offset + (16d × i), where i=0d to 1d



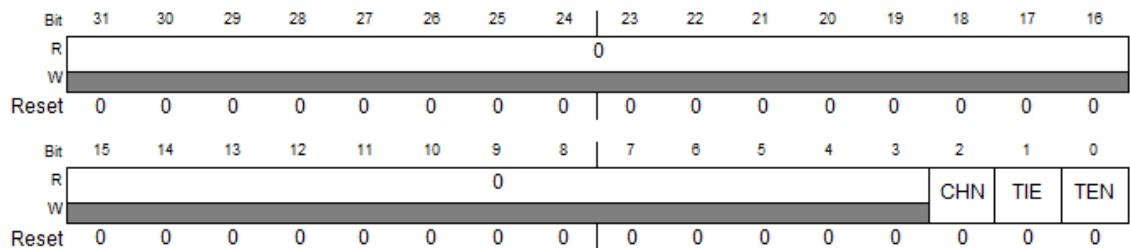
2.6.12. Descrierea PIT_CVALn

Domeniul	Descrierea
31–0 TVL	Valoarea curentă a timer-ului Reprezintă valoarea curentă a timer-ului, atunci când este activat. NOTĂ: • dacă timer-ul este dezactivat, a nu se folosi acest domeniu ca valoare sigură. • dacă timer-ul utilizează un numărător în jos, valoarea timer-ului este blocată în modul Debug dacă MCR[FRZ] este setat .

2.6.13. Registrul de control al timer-ului (PIT_TCTRLn)

Acești regiștri conțin biții de control pentru fiecare timer.

Address: 4003_7000h base + 108h offset + (16d × i), where i=0d to 1d



2.6.14. Descrierea PIT_TCTRLn

Domeniul	Descrierea
31–3 Reserved	Acest domeniu este rezervat Acest domeniu doar de citire este rezervat si are mereu valoarea 0
2 CHN	Modul de inlantuire Când este activat, timer-ul n-1 trebuie sa expire inainte ca timerul n sa fie decrementat cu 11. Timer 0 nu poate fi inlantuit.. 0 Timer nu este legat. 1 Timer is este legat ded timer-ul anterior. De exemplu, pentru canalul 2, dacă acest domeniu este setat, Timer 2 este legat de Timer 1.

1 TIE	Activarea timer-ului de intreruperi Când o întrerupere este în așteptare, sau, TFLGn[TIF] este setat, activând întreruperea va cauza imediat un eveniment de întrerupere. Pentru a evita acest lucru, TFLGn[TIF] asociat trebuie să fie mai întâi sters. 0 cererea de întrerupere de la timer-ul n este dezactivată 1 întreruperea va fi cerută atunci când TIF este setat.
0 TEN	Timer activat Activarea sau dezactivarea timer-ului. 0 Timer n este dezactivat 1 Timer n este activat.

2.6.15. Registrul de indicatori ai timer-ului (PIT_TFLGn)

Acești regiștrii conțin indicatorii de întrerupere ai PIT-ului.

Address: 4003_7000h base + 10Ch offset + (16d × i), where i=0d to 1d

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
R	0															
W																
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
R	0															TIF
W																w1c
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

2.6.16. Descrierea PIT_TFLGn

Domeniul	Descrierea
31–1 Reserved	Acest domeniu este rezervat Acest domeniu doar de citire este rezervat și are mereu valoarea 0
0 TIF	Indicatorul de întrerupere al timer-ului Setează în 1 la sfârșitul perioadei timer-ului. Scrierea unui 1 în acest indicator îl șterge. Scrierea unui 0 nu are nici un efect. Dacă este activat, sau, atunci când TCTRLn [TIE] = 1, TIF determină o cerere de întrerupere. 0 Timeout încă nu a avut loc. 1 Timeout a avut loc.

2.6.17. Operarea generală

Această secțiune oferă informații detaliate despre funcționarea internă a modulului. Fiecare timer-ul poate fi folosit pentru a genera impulsuri de declanșare și de întreruperi. Fiecare întrerupere este disponibilă pe o linie separată întrerupere.

2.6.18. Timere

Timerele generează declanșări și interale periodice, atunci când sunt activate. Timerele încărca valorile de pornire specificate în registrele lor LDVAL, numără în jos până la 0 și apoi încarcă din nou valoarea de start respectivă. De fiecare dată când un timer ajunge la 0, se va genera un impuls de declanșare și setează un indicatorul de întrerupere.

Toate întreruperile pot fi activate sau mascate prin setarea TCTRLn [TIE]. O nouă întrerupere poate fi generată numai după ce precedentă este terminată. Dacă se dorește, valoarea curentă numărată a timer-ului poate fi citită prin intermediul registrului CVAL. Perioada de numărare poate fi repornită, mai întâi prin dezactivare și apoi prin activarea timerului cu TCTRLn [10]. A se vedea figura de mai jos.

2.6.19. Oprirea și pornirea unui timer

Perioada de numărare a unui timer aflat în funcțiune poate fi modificată mai întâi prin dezactivarea timer-ului, apoi prin setarea unei noi valori de încărcare și apoi prin activarea din nou a timer-ului. A se vedea figura de mai jos.

2.6.20. Modificarea perioadei de rulare a timer-ului

De asemenea, este posibilă modificarea perioadei de numărare fără a reporni timerul prin scrierea LDVAL cu noua valoare de încărcare. Această valoare va fi apoi încărcată după următorul eveniment de declanșare. Vezi figura de mai jos

2.6.21. Modul Debug

În modul Debug, timerul va fi blocat pe baza MCR [FRZ]. Acest lucru este destinat dezvoltării de software, care să permită dezvoltatorului să oprească procesorul, să investigheze starea actuală a sistemului, de exemplu, valorile timer, și apoi să continue operațiunea.

2.6.22. Interuperi

Toate timerele suportă generare de întreruperi. Întreruperile timer-ului pot fi activate prin setarea TCTRLn [TIE]. TFLGn [TIF] sunt setate în 1 atunci când un timeout apare pe timerul asociat, și sunt transformate în 0 la scrierea unui 1 la TFLGn [TIF] corespunzător.

2.6.23. Timerele înlănțuite

Atunci când un timer are modul de înlănțuire activat, acesta va conta numai după ce timerul anterior a expirat. Deci dacă timerul n-1 a numărat până la 0, timerul n va decrement valoarea cu unu. Acest lucru permite înlănțuirea anumitor timeri împreună pentru a forma un timer mai mare. Acest prim timer (timer 0) nu poate fi legat de orice alte timer.

3. Deșfășurarea lucrării

3.1. Probleme rezolvate

3.1.1. Exemplu de configurare

În configurația de exemplu: • PIT ceas are o frecvență de 50 MHz. • Timer 1 creează o întrerupere fiecare ms 5.12. • Timer 3 creează un eveniment de declanșare fiecare 30 ms. Modulul de PIT trebuie activat de scris un 0 la MCR. Ceasul de 50 MHz frecvență echivaleaza cu o perioadă de ceas de 20 ns. Cronometru 1 are nevoie pentru a declanșa fiecare ns $5.12 \text{ ms}/20 = 256,000$ cicluri și Timer 3 fiecare 30 ms/20 ns = 1.500.000 de cicluri. Valoarea pentru LDVAL registru declanșare este calculat ca: LDVAL de declanșare = (perioada / ceas perioada) -1 Acest lucru înseamnă LDVAL1 și LDVAL3 trebuie să fie scris cu 0x0003E7FF și 0x0016E35F respectiv. Întrerupere pentru Timer 1 este activat prin setarea TCTRL1 [TIE]. Timer-ul este pornit de scris 1 la TCTRL1 [10]. Cronometru 3 se utilizează numai pentru declanșarea. Prin urmare, Timer 3 este început de scris un 1 la TCTRL3 [10]. TCTRL3 [TIE] rămâne la 0. Următorul exemplu de cod meciuri setup descris: <pre>// turn on PIT PIT_MCR = 0x00; // Timer 1 PIT_LDVAL1 = 0x0003E7FF; // setup timer 1 for 256000 cycles</pre>	În configurația de exemplu: • PIT ceas are o frecvență de 50 MHz. • Timer 1 creează o întrerupere fiecare ms 5.12. • Timer 3 creează un eveniment de declanșare fiecare 30 ms. Modulul de PIT trebuie activat de scris un 0 la MCR Ceasul de 50 MHz frecvență echivaleaza cu o perioadă de ceas de 20 ns. Cronometru 1 are nevoie pentru a declanșa fiecare ns $5.12 \text{ ms}/20 = 256,000$ cicluri și Timer 3 fiecare 30 ms/20 ns = 1.500.000 de cicluri. Valoarea pentru LDVAL registru declanșare este calculat ca: LDVAL de declanșare = (perioada / ceas perioada) -1 Acest lucru înseamnă LDVAL1 și LDVAL3 trebuie să fie scris cu 0x0003E7FF și 0x0016E35F respectiv. Întrerupere pentru Timer 1 este activat prin setarea TCTRL1 [TIE]. Timer-ul este pornit de scris 1 la TCTRL1 [10]. Cronometru 3 se utilizează numai pentru declanșarea. Prin urmare, Timer 3 este început de scris un 1 la TCTRL3 [10]. TCTRL3 [TIE] rămâne la 0. Următorul exemplu de cod meciuri setup descris: <pre>// turn on PIT PIT_MCR = 0x00; // Timer 2 PIT_LDVAL2 = 0x00000009; // setup Timer 2 for</pre>
--	---

PIT_TCTRL1 = TIE; // enable Timer 1 interrupts PIT_TCTRL1 = TEN; // start Timer 1 // Timer 3 PIT_LDVAL3 = 0x0016E35F; // setup timer 3 for 1500000 cycles PIT_TCTRL3 = TEN; // start Timer 3	10 counts PIT_TCTRL2 = TIE; // enable Timer 2 interrupt PIT_TCTRL2 = CHN; // chain Timer 2 to Timer 1 PIT_TCTRL2 = TEN; // start Timer 2 // Timer 1 PIT_LDVAL1 = 0x23C345FF; // setup Timer 1 for 600 000 000 cycles PIT_TCTRL1 = TEN; // start Timer 1
--	---

3.1.2. Să se aprindă un led folosind timerul PIT, atât cu intrerupere cât și fără.

Problema 1	Problema 2
<pre> #include "derivative.h" /* include peripheral declarations */ typedef unsigned long uint32_t; #define SIM_SCGC5 (*((uint32_t*)0x40048038u)) #define SIM_SCGC6 (*((uint32_t*)0x4004803Cu)) #define PORTE_PCR29 (*((uint32_t*)0x4004D074u)) #define PORTE_PCR31 (*((uint32_t*)0x4004D07Cu)) #define GPIOE_PDDR (*((uint32_t*)0x400FF114u)) #define GPIOE_PCOR (*((uint32_t*)0x400FF108u)) #define GPIOE_PSOR (*((uint32_t*)0x400FF104u)) #define GPIOE_PTOR (*((uint32_t*)0x400FF10Cu)) #define PIT_MCR (*((uint32_t*)0x40037000u)) </pre>	<pre> #include "derivative.h" /* include peripheral declarations */ typedef unsigned long uint32_t; #define SIM_SCGC5 (*((uint32_t*)0x40048038u)) #define SIM_SCGC6 (*((uint32_t*)0x4004803Cu)) #define PORTE_PCR29 (*((uint32_t*)0x4004D074u)) #define PORTE_PCR31 (*((uint32_t*)0x4004D07Cu)) #define GPIOE_PDDR (*((uint32_t*)0x400FF114u)) #define GPIOE_PCOR (*((uint32_t*)0x400FF108u)) #define GPIOE_PSOR (*((uint32_t*)0x400FF104u)) #define GPIOE_PTOR (*((uint32_t*)0x400FF10Cu)) #define PIT_MCR (*((uint32_t*)0x40037000u)) </pre>

<pre> #define PIT_LDVAL0 *((uint32_t*)0x40037100u)) #define PIT_TCTRL0 *((uint32_t*)0x40037108u)) #define PIT_TFLG0 *((uint32_t*)0x4003710Cu)) #define NVIC_ISER *((uint32_t*)0xE000E100u)) void wait250ms() { asm(" push {R1-R3} \n"); asm(" movs R1, #250 \n"); asm(" loop1: \n"); asm(" movs R2, #48 \n"); asm(" loop2: \n"); asm(" movs R3, #141 \n"); asm(" loop3: \n"); asm(" sub R3, R3, #1 \n"); asm(" bne loop3 \n"); asm(" sub R2, R2, #1 \n"); asm(" bne loop2 \n"); asm(" sub R1, R1, #1 \n"); asm(" bne loop1 \n"); asm(" pop {R1-R3} \n"); } void PIT_IRQHandler() { // sterg flag-ul PIT_TFLG0 = 0x00000001u; // modific starea LED-ului GPIOE_PTOR = 0x20000000u; </pre>	<pre> #define PIT_LDVAL0 *((uint32_t*)0x40037100u)) #define PIT_TCTRL0 *((uint32_t*)0x40037108u)) #define PIT_TFLG0 *((uint32_t*)0x4003710Cu)) #define NVIC_ISER *((uint32_t*)0xE000E100u)) void wait250ms() { asm(" push {R1-R3} \n"); asm(" movs R1, #250 \n"); asm(" loop1: \n"); asm(" movs R2, #48 \n"); asm(" loop2: \n"); asm(" movs R3, #141 \n"); asm(" loop3: \n"); asm(" sub R3, R3, #1 \n"); asm(" bne loop3 \n"); asm(" sub R2, R2, #1 \n"); asm(" bne loop2 \n"); asm(" sub R1, R1, #1 \n"); asm(" bne loop1 \n"); asm(" pop {R1-R3} \n"); } // fara intreruperi int main(void) { // da clock la port E SIM_SCGC5 = 0x00002182u; </pre>
--	--

<pre> // si a semnalului de pe difuzor GPIOE_PTOR = 0x80000000u; } // cu intrerupere PIT int main(void) { // da clock la port E SIM_SCGC5 = 0x00002182u; // mux-eaza pinul 29 din portul E (conectat la LED2 (rosu)) ca GPIO PORTE_PCR29 = 0x00000140u; // si pinul 31 (conectat la pinul 1 din conectorul J3) PORTE_PCR31 = 0x00000140u; // si programeaza-le ca iesire GPIOE_PDDR = 0x20000000u; GPIOE_PDDR = 0x80000000u; // aprinde LED-ul rosu GPIOE_PCOR = 0x20000000u; // asteapta o secunda wait250ms(); wait250ms(); wait250ms(); wait250ms(); // stinge LED-ul rosu GPIOE_PSOR = 0x20000000u; // asteapta o secunda wait250ms(); wait250ms(); </pre>	<pre> // mux-eaza pinul 29 din portul E (conectat la LED2 (rosu)) ca GPIO PORTE_PCR29 = 0x00000140u; // si programeaza-l ca iesire GPIOE_PDDR = 0x20000000u; // aprinde LED-ul rosu GPIOE_PCOR = 0x20000000u; // asteapta o secunda wait250ms(); wait250ms(); wait250ms(); wait250ms(); // stinge LED-ul rosu GPIOE_PSOR = 0x20000000u; // asteapta o secunda wait250ms(); wait250ms(); wait250ms(); wait250ms(); // Programeaza PIT-ul: // da-i clock SIM_SCGC6 = 0x00800000u; // activeaza klok-ul PIT_MCR = 0x00000000u; // seteaza-i constanta de divizare pentru canalul 0 PIT_LDVAL0 = 0x01388000u; // 0.5Hz PIT_LDVAL0 = 0x004E2000u; // 2Hz </pre>
--	--

<pre> wait250ms(); wait250ms(); // Programeaza PIT-ul: // da-i clock SIM_SCGC6 = 0x00800000u; // activeaza klok-ul PIT_MCR = 0x00000000u; // seteaza-i constanta de divizare pentru canalul 0 PIT_LDVAL0 = 0x01388000u; // 0.5Hz //PIT_LDVAL0 = 0x004E2000u; // 2Hz //PIT_LDVAL0 = 0x00002800u; // 1000Hz // porneste canalul 0, cu intreruperi PIT_TCTRL0 = 0x00000003u; // sterg flag-ul PIT_TFLG0 = 0x00000001u; //// activez global intreruperile <-- NU E NEVOIE! //asm("CPSIE i"); // se pare ca sunt activate by default // activez intreruperea de PIT din NVIC NVIC_ISER = 0x00400000; while(1) { } return 0; } </pre>	<pre> // porneste canalul 0, fara intreruperi PIT_TCTRL0 = 0x00000001u; // porneste canalul 0, cu intreruperi //PIT_TCTRL0 = 0x00000003u; // resetez flag-ul //PIT_TFLG0 = 0x00000001u; while(1) { // verific flag-ul de terminare a numararii if ((PIT_TFLG0 & 0x00000001u) != 0) { // modific starea LED-ului GPIOE_PTOR = 0x20000000u; // resetez flag-ul PIT_TFLG0 = 0x00000001u; } } return 0; } </pre>
--	---

3.2. Probleme propuse

3.2.1. Să se aprinda LED-ul verde folosind timerul PIT fără intrerupere

3.2.2. Să se aprinda LED-ul verde folosind timerul PIT cu intrerupere

GENERAREA DE SEMNALE DIGITALE

1. Scopul lucrării

Lucrarea își propune deprinderea de către studenți a modului în care microcontrolerul ARM Cortex M0+ poate fi programat pentru a genera diverse tipuri de semnale digitale (semnale PWM, note muzicale, secvențe pseudoaleatoare)

2. Breviar teoretic

2.1. Semnale PWM

2.1.1. Informații generale

Denumirea PWM este un acronim ce vine de la “pulse with modulation”. Cu alte cuvinte, un semnal PWM este un semnal cu modulare a duratei pulsurilor.

Mai precis, un semnal PWM este un semnal digital de o frecvență fixată, al cărui singur parametru variabil este o durată pulsului (deci lungimea intervalului de timp cât semnalul stă în “1” logic pe timpul unei perioade a semnalului).

Figura următoare ilustrează două semnale PWM. Ambele au aceeași perioadă T , dar duratele pulsurilor (W_1 și W_2) sunt diferite.

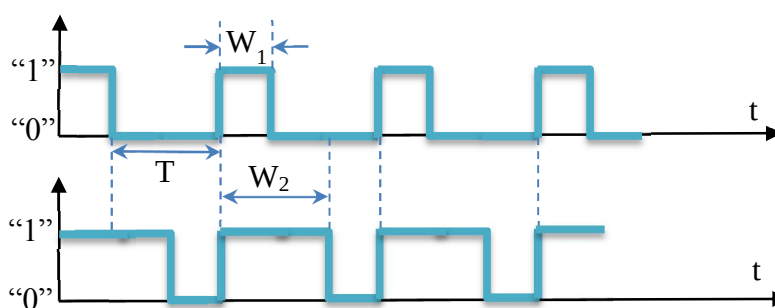


Figura 1. Semnale PWM

Raportul dintre durată pulsului și perioada semnalului PWM este parametrul esențial ce caracterizează un astfel de semnal și poartă numele de factor de umplere:

Cum W poate fi minim 0 și maxim T , rezultă că . (Observație: Dacă semnalul este un “0” logic constant în timp, iar dacă 1 semnalul este “1” logic constant în timp.)

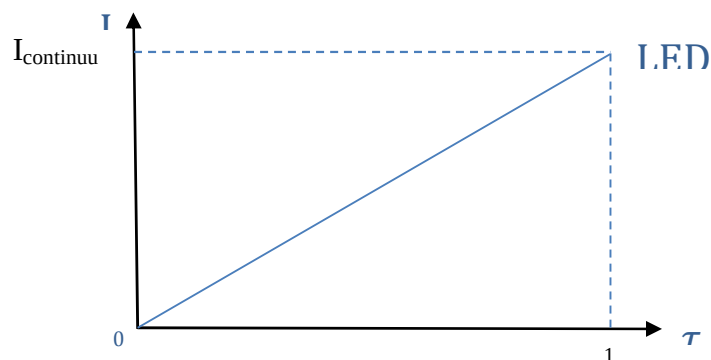
Graficul semnalelor PWM este dată în mare parte de caracteristica unor sisteme fizice de a avea inerție. Dacă privim un LED care se aprinde și se stinge din ce în ce mai repede, de la un punct în acolo (și anume, peste o frecvență de 45-50 Hz), nu mai distingem aprinderile și stingerile, ci avem iluzia că LED-ul ar fi aprins continuu. Intensitatea la care percepem LED-ul ca fiind aprins continuu este cu atât mai mare cu cât LED-ul este aprins mai mult timp pe durata unei perioade a semnalului PWM.

Cu alte cuvinte, intensitatea la care percepem LED-ul este direct proporțională cu factorul de umplere. Figura următoare ilustrează această afirmație.

În mod similar putem folosi semnalele PWM pentru a varia turația unui motor de curent continuu sau pentru a varia debitul printr-o conductă folosind o electrovalvă.

3.1.1. Despre generarea semnalelor PWM

Microcontroller-ul Cortex M0+ KL46Z conține un modul specializat pentru generarea de semnal PWM. Acest modul poate fi configurat și programat cu ușurință folosind Processor Expert.



Pentru a înțelege mai bine semnalele PWM ne vom ocupa aici și de generarea lor în manieră software, folosind un timer și întreruperi.

În acest caz trebuie să alegem numărătorul de valori discrete pe care le va putea avea factorul de umplere. Acesta va fi și numărătorul de valori distincte pe care le va putea avea pulsul pe durata unei perioade.

De exemplu, dacă dorim să putem varia pe tau în 10 trepte egale între 0 și 1, atunci soluția constă în a diviza o perioadă în 10 intervale de timp egale. Acele momente de timp vor fi marcate prin cereri de întrerupere către procesor, iar în subrutinele de tratare a lor vom decide în funcție de factorul de umplere ales și de valoarea unui contor (pe care îl vom reseta la fiecare nou început de perioadă) dacă să ținem semnalul în "0" sau în "1". Cele explicate aici sunt ilustrate în figura următoare:

Un exemplu practic de generare a semnalului PWM pentru comanda unui LED va fi arătat în secțiunea de probleme rezolvate.

2.2.Sunete

2.2.1. Informații generale

Un sunet este o vibrație a aerului ce poate fi percepută de către urechea umană. Prin urmare, frecvența acestei vibrații trebuie să fie cuprinsă între 20Hz și 20KHz

2.2.2. Generarea notelor muzicale

Valorile frecvențelor pentru notele muzicale sunt standardizate. De exemplu, pentru nota “la” frecvența este de 40Hz, iar celelalte note se obțin urcând sau scăzând cu cate un ton/semiton

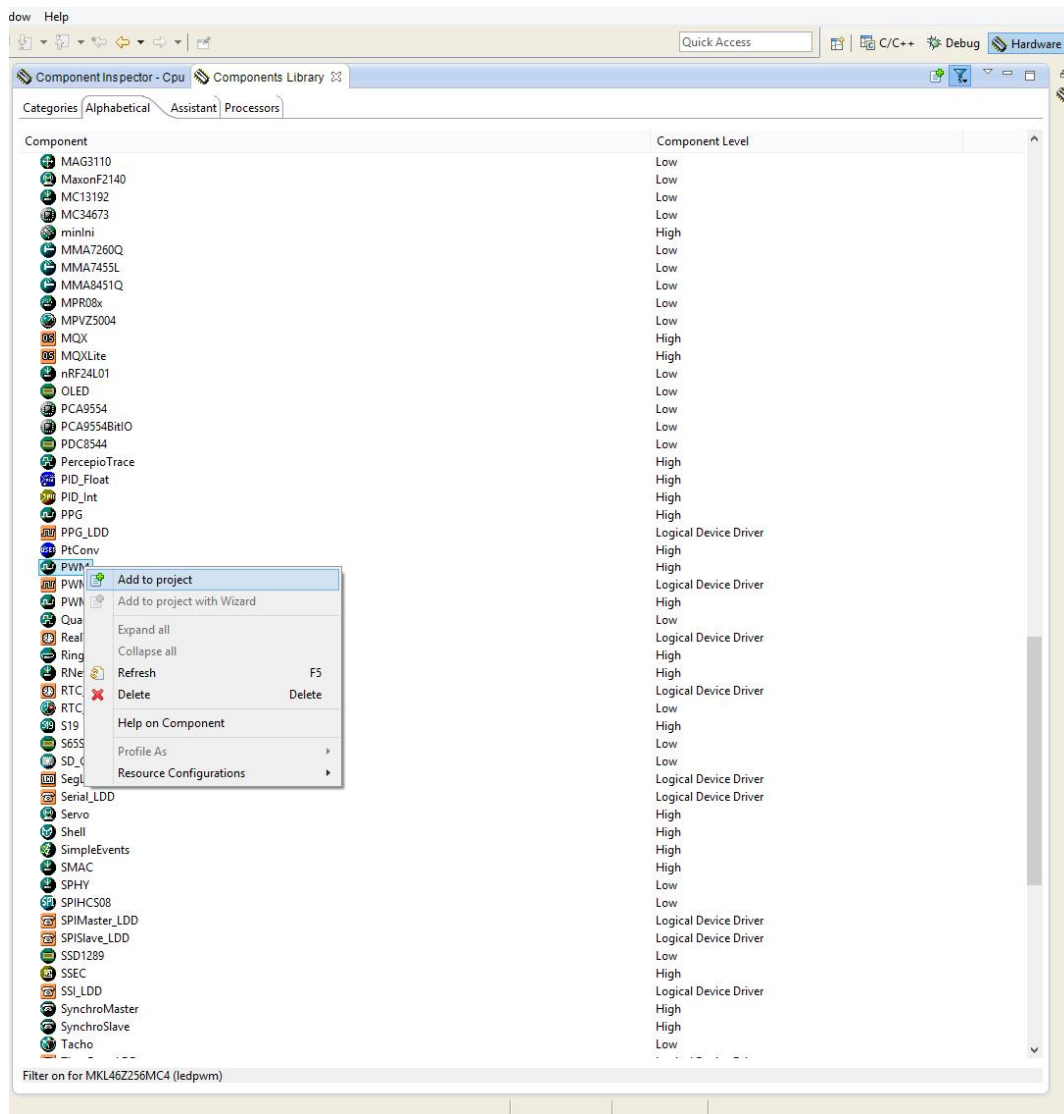
Bineînțeles că nu doar frecvența semnalului are o influență asupra senzației auditive percepute, ci și amplitudinea (sau forma) semnalului. Cum ne ocupăm acum doar de generarea de semnale digitale, nu avem foarte mare libertate la acest capitol: semnalul va fi jumatate din perioada sa “1” și cealaltă jumătate “0”. În felul acesta apropiindu-ne cel mai mult de forma unei sinsoide.

3. Desfășurarea lucrării

3.1. Probleme rezolvate

3.1.1. Folosind Processor Expert sa se aprindă ledurile de pe placa de dezvoltare KL46Z, în PWM, cel roșu având factorul de umplere 1/4, iar cel verde 3/4.

După configurarea CodeWarrior pentru a lucra cu Processor Expert vom adauga componenta PWM si o vom configura atat pentru LED-ul roșu cât și pentru LED-ul verde.



C/C++ - LED_PWM/Sources/ProcessorExpert.c - CodeWarrior Development Studio

File Edit Source Refactor Search Project MQX Tools Processor Expert Run Window Help

CodeWarrior Projects

File Name Build

- acelerometru
- asm_test
- JingleBells
- led
- LED_PWM : FLASH
 - Binaries
 - Documentation
 - FLASH
 - Generated_Code
 - ProcessorExpert.pe
 - Project-Headers
 - Project_Settings
 - Sources
 - ledpwm : FLASH
 - PIT_Demo
 - senzor
 - test
 - ToggleLED

Components - LED_PWM

- Generator_Configurations
- FLASH
- OSs
- Processors
- Cpu:MKL46Z256VMC4
- Components
 - Referenced_Components
 - PWM1:PWM
 - PWM2:PWM
- PDD

Component Inspector - PWM1

Properties Methods Events

Name	Value	Details
Component name	PWM1	
PWM or PPC device	TPM0_C2V	TPM0_C2V
Duty compare		
Output pin	CMPO_IN5/ADCO_SE4b/PT29/TPM0_CH2/TPM_CLKIN0	CMPO_IN5/ADCO_SE4b/PT29/TPM0_CH2/TPM_CLKIN0
Output pin signal		
Counter	TPM0_CNT	TPM0_CNT
Interrupt service/event	Disabled	
Period	400 ms	400 ms
Starting pulse width	100 ms	100 ms
Initial polarity	low	
Same period in modes	no	
Component uses entire timer	no	
Initialization		
Enabled in init. code	yes	
Events enabled in init.	yes	
CPU clock/speed selection		
High speed mode	This component enabled	This component is enabled
Low speed mode	This component disabled	This component is disabled
Slow speed mode	This component disabled	This component is disabled
Referenced components		

Problems Console

0 errors, 2 warnings, 0 others

Description

Warnings (2 items)

Resource

C/C++ - LED_PWM/Sources/ProcessorExpert.c - CodeWarrior Development Studio

File Edit Source Refactor Search Project MQX Tools Processor Expert Run Window Help

CodeWarrior Projects

File Name Build

- acelerometru
- asm_test
- JingleBells
- led
- LED_PWM : FLASH
 - Binaries
 - Documentation
 - FLASH
 - Generated_Code
 - ProcessorExpert.pe
 - Project-Headers
 - Project_Settings
 - Sources
 - ledpwm : FLASH
 - PIT_Demo
 - senzor
 - test
 - ToggleLED

Components - LED_PWM

- Generator_Configurations
- FLASH
- OSs
- Processors
- Cpu:MKL46Z256VMC4
- Components
 - Referenced_Components
 - PWM1:PWM
 - PWM2:PWM
- PDD

Component Inspector - PWM2

Properties Methods Events

Name	Value	Details
Component name	PWM2	
PWM or PPC device	TPM0_C5V	TPM0_C5V
Duty compare		
Output pin	LCD_P45/ADCO_SE6b/PTD5/SPI1_SCK/UART2_TX/TPM0_CH5	LCD_P45/ADCO_SE6b/PTD5/SPI1_SCK/UART2_TX/TPM0_CH5
Output pin signal		
Counter	TPM0_CNT	TPM0_CNT
Interrupt service/event	Disabled	
Period	400 ms	400 ms
Starting pulse width	300 ms	300 ms
Initial polarity	low	
Same period in modes	no	
Component uses entire timer	no	
Initialization		
Enabled in init. code	yes	
Events enabled in init.	yes	
CPU clock/speed selection		
High speed mode	This component enabled	This component is enabled
Low speed mode	This component disabled	This component is disabled
Slow speed mode	This component disabled	This component is disabled
Referenced components		

Problems Console

0 errors, 2 warnings, 0 others

Description

Warnings (2 items)

Resource

3.1.2. Exemplu problema sunete

```
/*
 * Aplicatie ce canta "Jingle Bells" pe LED-ul rosu si
 * pe un difuzor conectat cu + la pinul 1 din J3 si cu
 * - la pinul 14 din J3.
 */

//#include "derivative.h" /* include peripheral declarations */

// --> Pentru accesare registri module periferice

typedef unsigned long uint32_t;

#define SIM_SCGC5 (*(uint32_t*)0x40048038u)

#define PORTC_PCR3 (*(uint32_t*)0x4004B00Cu)

#define PORTE_PCR29 (*(uint32_t*)0x4004D074u)
#define PORTE_PCR31 (*(uint32_t*)0x4004D07Cu)

#define GPIOC_PDDR (*(uint32_t*)0x400FF094u)
#define GPIOC_PDIR (*(uint32_t*)0x400FF090u)

#define GPIOE_PDDR (*(uint32_t*)0x400FF114u)
#define GPIOE_PCOR (*(uint32_t*)0x400FF108u)
#define GPIOE_PSOR (*(uint32_t*)0x400FF104u)
#define GPIOE_PTOR (*(uint32_t*)0x400FF10Cu)

// --> Pentru muzica
```

```

// perioade note, in microsecunde (us)
int T[] = {3823, 3608, 3405, 3214, 3034, 2864, 2703,
           2551, 2408, 2273, 2145, 2025, 1911};

// durate note, in microsecunde (us)
//#define R 250000u
//#define R 125000u
#define R 160000
#define D1 (4*R)
#define D2 (2*R)
#define D4 R
#define D8 (R/2)
#define D16 (R/4)
#define D4p (D4+D4/2)
#define D2p (D2+D2/2)

// --> Partitura "Jingle Bells"
int NN = 78;
int nota[] = {0, 9, 7, 5, 0, 0, 0, 0, 9, 7, 5, 2, -1, 2, 11, 9, 7, 4, -1,
              12, 12, 11, 7, 9, -1, 0, 9, 7, 5, 0, -1, 0, 9, 7, 5,
              2, 2, 2, 11, 9, 7, 12, 12, 12, 12, 12, 12, 11, 7, 5, 12,
              9, 9, 9, 9, 9, 9, 9, 9, 12, 5, 7, 9, -1, 11, 11, 11, 11,
              11, 9, 9, 9, 9, 9, 7, 7, 9, 7, 12};

int durata[] = {D4, D4, D4, D4, D2p, D8, D8, D4, D4, D4, D4, D2p, D4, D4, D4, D4,
                D4, D4, D4, D4, D2p, D4, D4, D4, D4, D4, D2p, D4, D4, D4, D4,
                D2p, D4, D4, D4, D4, D4, D4, D4, D4, D4, D4, D4, D4, D4, D2, D2,
                D4, D4, D2, D4, D4, D2, D4, D4, D4p, D8, D2p, D4, D4, D4, D4p,
                D8,
                D4, D4, D4, D8, D8, D4, D4, D4, D4, D4, D2, D2};

// --> Functii ajutatoare

```



```

void Asteapta_us(unsigned long nr_us);

int main(void)
{
    // --> Programeaza pinii pentru LED si pentru difuzor

    // da clock la porturile E si C
    SIM_SCGC5 = 0x00002982u;
    // mux-eaza pinul 29 din portul E (conectat la LED2 (rosu)) ca GPIO
    PORTE_PCR29 = 0x00000140u;
    // si pinul 31 (conectat la pinul 1 din conectorul J3; nota: pinul 14 e GND)
    PORTE_PCR31 = 0x00000140u;
    // si programeaza-le ca iesire
    GPIOE_PDDR |= 0x20000000u;
    GPIOE_PDDR |= 0x80000000u;
    // mux-eaza pinul 3 din portul C (conectat la butonul SW1) ca GPIO
    // (si activeaza rezistor de pullup)
    PORTC_PCR3 = 0x00000103u;
    // si programeaza-l ca intrare
    GPIOC_PDDR &= ~0x00000008u;

    // --> Initializeaza starea LED-ului si a difuzorului

    // aprinde LED-ul rosu
    GPIOE_PCOR |= 0x20000000u;
    // si semnalul de pe difuzor pune-l pe 1
    GPIOE_PSOR |= 0x80000000u;

    // asteapta 1 secunda
    Asteapta_us(1000000);

    // stinge LED-ul rosu

```

```

GPIOE_PSOR |= 0x20000000u;
// si semnalul de pe difuzor pune-l pe 0
GPIOE_PCOR |= 0x80000000u;

// --> Bucla infinita

while(1)
{
    // daca butonul SW1 e apasat
    if ( (GPIOC_PDIR & 0x00000008u) == 0 )
    {
        // canta "Jingle Bells"

        int i, j;
        // DFZ <- 0
        GPIOE_PCOR |= 0x80000000u;
        GPIOE_PSOR |= 0x20000000u;//LED
        for (i=0; i<NN; i++)
        {
            if (nota[i] >= 0) // daca este nota
            {
                int DN = (durata[i])/T[nota[i]];
                for (j=0; j<DN; j++)
                {
                    // DFZ <- 1
                    GPIOE_PSOR |= 0x80000000u;
                    GPIOE_PCOR |= 0x20000000u;//LED
                    Asteapta_us(T[nota[i]]/2);
                    // DFZ <- 0
                    GPIOE_PCOR |= 0x80000000u;
                    GPIOE_PSOR |= 0x20000000u;//LED
                    Asteapta_us(T[nota[i]]/2);
                }
            }
        }
    }
}

```

```

        }

    }

    else // daca este pauza
    {
        Asteapta_us(durata[i]);
    }

    // asteapta intre note
    Asteapta_us(D8);
}

}

}

return 0;
}

// --> Corp functii ajutatoare
void Asteapta_us(unsigned long nr_us)
{
    while (nr_us>0)
    {
        // asteapta 1 us (aproximativ)
        asm("          push {R3}                \n");
        asm("          movs R3, #5              \n");
        asm("  loop3:                \n");
        asm("          sub R3, R3, #1          \n");
        asm("          bne loop3              \n");
        asm("          pop {R3}               \n");

        nr_us--;
    }
}

```

3.2. Probleme propuse

3.2.1. Folosind modulul PWM al microprocesorului CortexM0+ să se aprindă LED-ul roșu având factorul de umplere $1/3$.

3.2.2. Folosind modulul PWM al microprocesorului CortexM0+ să se aprindă LED-ul verde având factorul de umplere $2/3$.